# Aula 02 – Primeira Simulação

## Simulações Computacionais para o Ensino de Robótica Inteligente e Aprendizado por Reforço

Leonardo de Souza Gomes
Orientador: Prof. Dr. André Luiz C. Ottoni

Material adaptado do tutorial oficial Webots[1]
Edital PROGRAD nº 27, de 26 de maio de 2026
Universidade Federal de Ouro Preto

2026

# Aula 02 – Primeira Simulação

**Objetivo:** Familiarizar-se com a interface e os conceitos básicos do Webots.

Ao final, você terá criado uma simulação contendo:

- Uma arena retangular com chão e paredes
- Algumas caixas de madeira
- Um robô e-puck
- Um programa controlador que movimenta o robô

O tutorial em que essa aula se baseia, que pode ser encontrado em:
https://cyberbotics.com/doc/guide/tutorial-1-your-first-simulation-in-webots?tab-language=c.

# Iniciar o Webots

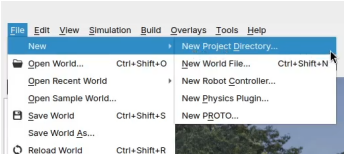
- Certifique-se de que o Webots está instalado seguindo as instruções oficiais de instalação.
- Inicie o Webots clicando duas vezes no ícone (ou via linha de comando).
- Na primeira execução, pode ser solicitado que você escolha um tema gráfico.
- Feche o *guided tour* caso ele seja exibido (ou aproveite para explorá-lo).

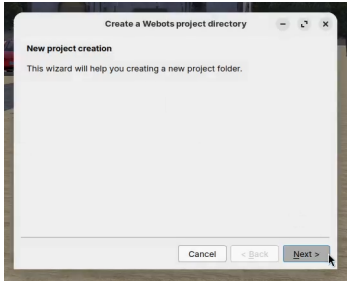
# Criar um Novo Mundo

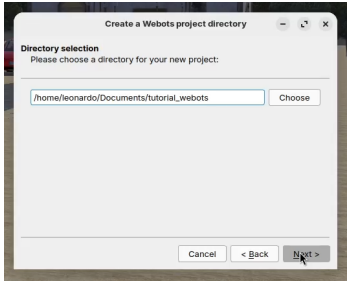
Pause a simulação atual clicando no botão de Pause da visualização 3D.

1. Vá em File > New Project Directory;
2. Nomeie o diretório do projeto: `tutorial_webots`;
3. Nomeie o mundo como: `tutorial_webots`;
4. Marque todas as caixas de seleção, incluindo *"Add a rectangle arena"*;
5. Clique em **Finish**.

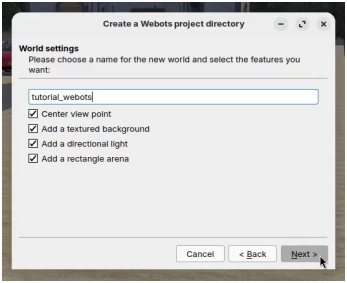
# Criar um Novo Mundo

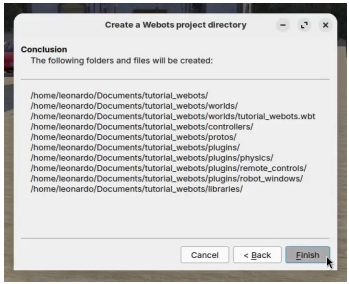


# Criar um Novo Mundo



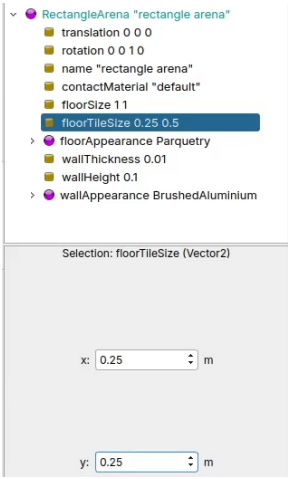




Agora com o projeto e um mundo Webots criado, podemos dar sequência ao tutorial.

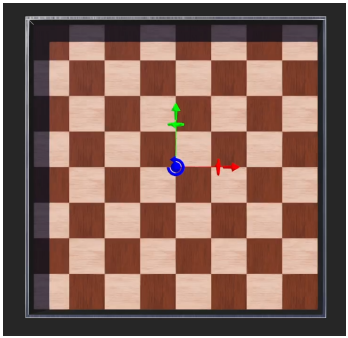
# Modificando a Arena

Modificando o quadriculado da arena.

1. Clique duas vezes sobre o objeto RectangleArena “rectangle arena”;
2. Clique duas vezes sobre o field “floorTileSize”;
3. Mude o valor x e y, de floorTileSize, de 0.5 e 0.5 para 0.25 e 0.25 respectivamente;



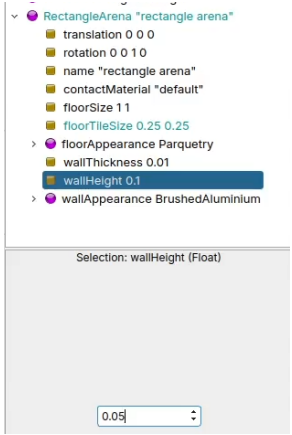

Resultado esperado:



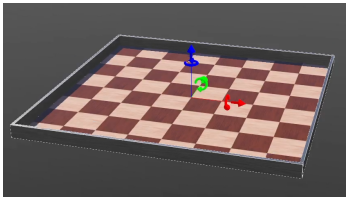
# Modificando a Arena

Podemos modificar também a altura da parede da nossa arena.

1. Clique duas vezes sobre o field "wallHeight";
2. Mude o valor de 0.1 para 0.05.



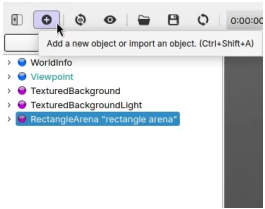

Resultado esperado:



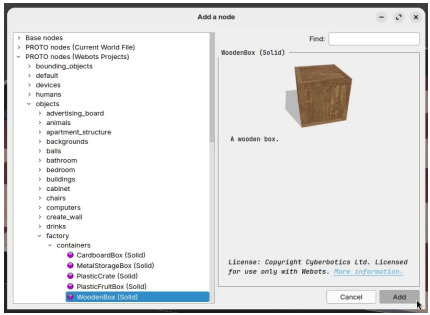
# Adicionando Objetos

Por padrão, o Webots oferece um conjunto de objetos para usarmos em nossos projetos. E para isso basta acessar o botão: "Add a new object or import an object. (Ctrl + Shift + A)".
Agora vamos adicionar uma caixa de madeira ao nosso mundo.

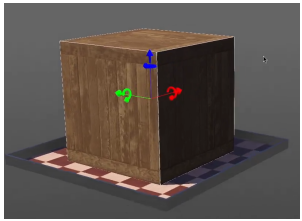
1. Clique no botão "Add a new object or import an object. (Ctrl + Shift + A)";




2. Selecione: PROTO nodes (Webots Projects) > objects > containers > WoodenBox (Solid) .



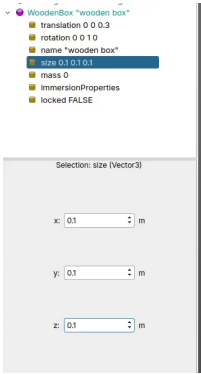

Resultado esperado:



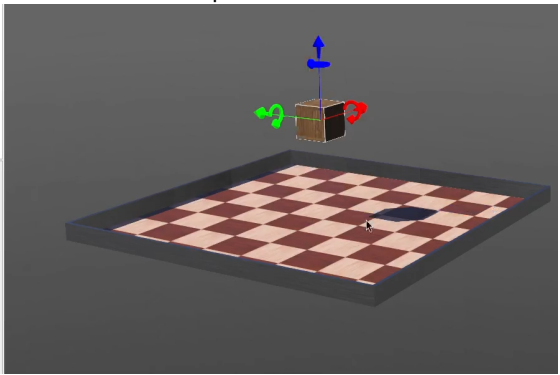
# Modificando o tamanho de objetos

A caixa de madeira adicionada anteriormente está ocupando muito espaço do nosso projeto. Para resolver isso vamos reduzir o seu tamanho.

1. Clique no field size e mude o x, y, z de 0.6, 0.6, 0.6 para 0.1, 0.1, 0.1 respectivamente.




Resultado esperado:



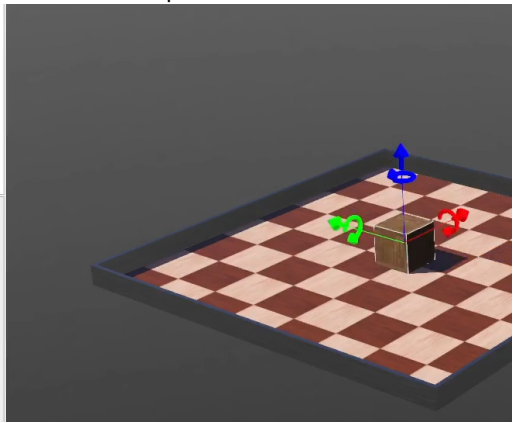
# Modificando a posição de objetos

Como visto anteriormente a nossa caixa está flutuando no espaço. Para resolver isso vamos mudar a sua posição.

1. Clique no translation e mude o x, y, z de 0.0, 0.0, 0.3 para 0.0, 0.0, 0.05 respectivamente.



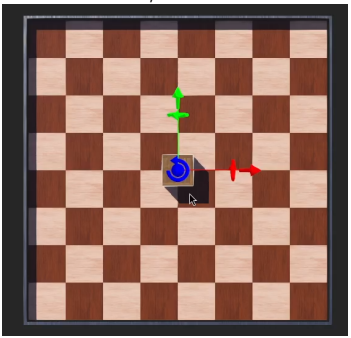

Resultado esperado:



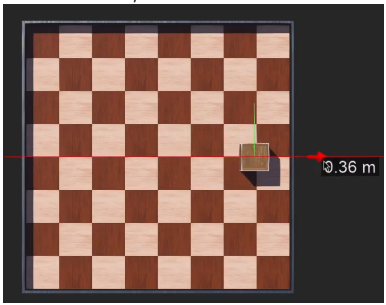
# Modificando a posição de objetos

Agora, vamos mudar a posição da caixa, porém de uma forma diferente. Colocaremos a caixa em um canto da arena.

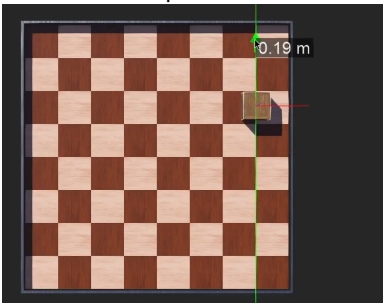
1. Clique na caixa de madeira;



3. Clique na seta reta e vermelha, e arraste para direita;




4. Clique na seta reta e verde, e arraste para cima.

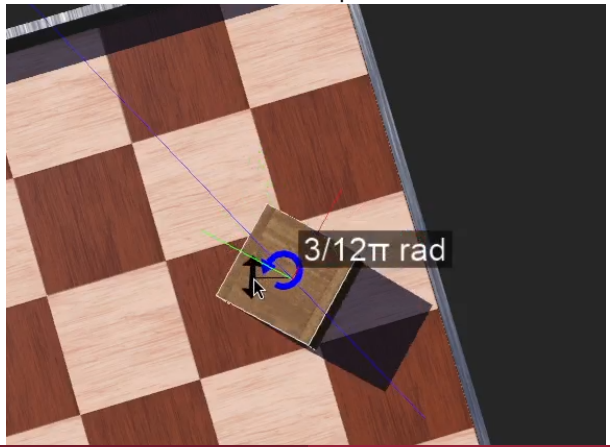


# Rotacionando a posição de objetos

Podemos mudar o ângulo de rotação dos nossos objetos ao clicar e arrastar as setas que circulam os eixos.

1. Clique na caixa de madeira;
2. Clique na seta azul que está circulando a seta reta do eixo Z (Azul);
3. Arraste o mouse e coloque a caixa na posição que preferir.

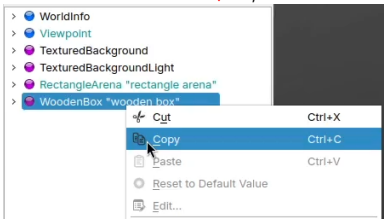
Resultado esperado:



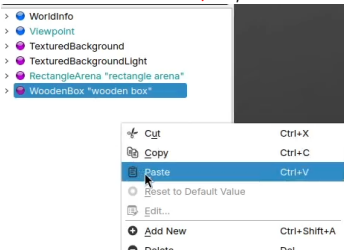
# Copiando de objetos

Outra funcionalidade que temos presente no Webots é a de copiar objetos. Adicione três novas caixas e espalhe-as pela arena, totalizando 4 caixas na arena.

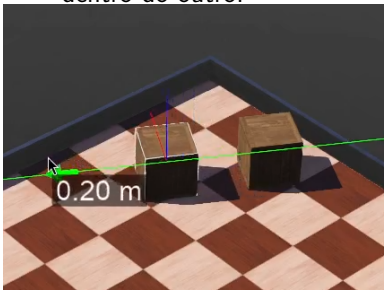
1. Copie o objeto da caixa de madeira Ctrl + c;




3. Cole o objeto da caixa de madeira Ctrl + v;



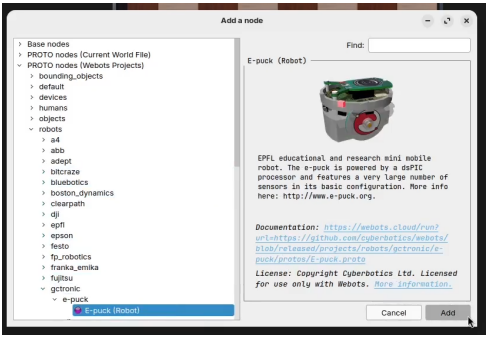

4. Clique e arraste novamente a caixa pois um objeto estará dentro do outro.



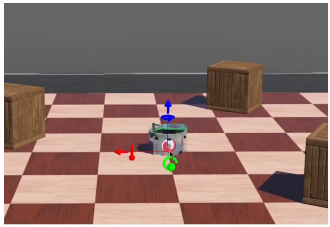
# Adicionar o Robô e-puck

**Regra importante antes de salvar: Pausar → Resetar → Modificar → Salvar**

1. Clique em **Add** → *PROTO nodes > robots > gctronic > e-puck > E-puck*;
2. Clique no botão de executar a simulação.




Resultado esperado:



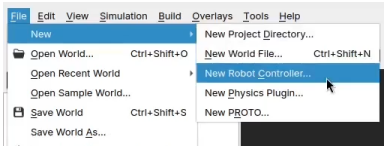
O e-puck é um robô diferencial compacto com rodas, LEDs e sensores (câmera, sensores de distância).

**Regra importante antes de salvar: Pausar → Resetar → Modificar → Salvar**

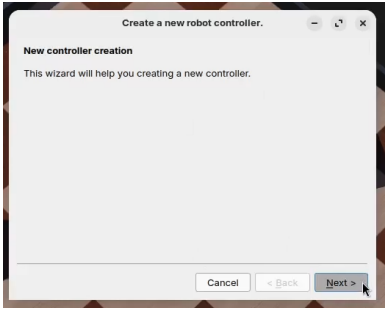
# Criando um controlador

Como nós já adicionamos um robô, vamos agora aprender a como programar as ações do robô. Para isso, nos temos utilizamos os controllers, que são códigos escritos em uma linguagem de programação, em nossos tutoriais, utilizaremos a linguagem C.

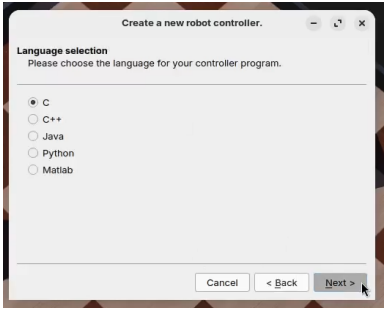
1. File > New > New Robot Controller...;




3. Clique em Next;

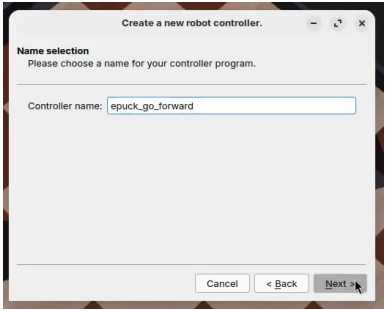



4. Clique em C > Next.

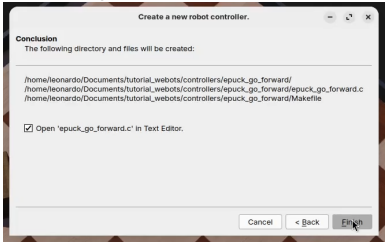


# Criando um controlador

4. Nomeie o controller como: `epuck_go_forward`;




5. Clique em Next.




Resultado esperado:

epuck_go_forward.c

```c
/*
 * File:          epuck_go_forward.c
 * Date:
 * Description:
 * Author:
 * Modifications:
 */

/*
 * You may need to add include files like <webots/distance_sensor.h> or
 * <webots/motor.h>, etc.
 */
#include <webots/robot.h>

/*
 * You may want to add macros here.
 */
#define TIME_STEP 64

/*
 * This is the main program.
 * The arguments of the main function can be specified by the
 * "controllerArgs" field of the Robot node
 */
int main(int argc, char **argv) {
  /* necessary to initialize webots stuff */
  wb_robot_init();

  /*
   * You should declare here WbDeviceTag variables for storing
   * robot devices like this:
   *  WbDeviceTag my_sensor = wb_robot_get_device("my_sensor");
   *  WbDeviceTag my_actuator = wb_robot_get_device("my_actuator");
   */

```

# Controlador: Movimento por Posição

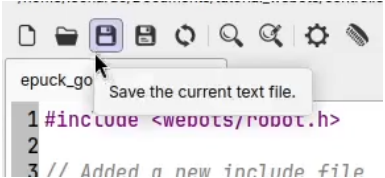
**Pause** a simulação e **substitua** o código do controlador pelo código abaixo:

```c
#include <webots/robot.h> // biblioteca padrao do Webots
#include <webots/motor.h> // funcoes de controle dos motores
#define TIME_STEP 64      // intervalo de tempo de cada passo (ms)

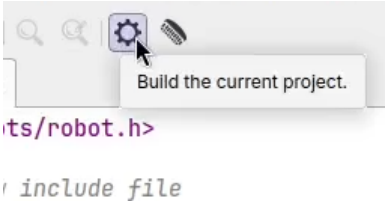
int main(int argc, char **argv) {
  wb_robot_init(); // inicializa o robo
  // motor esquerdo
  WbDeviceTag left_motor  = wb_robot_get_device("left wheel motor");
  // motor direito
  WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
  wb_motor_set_position(left_motor,  10.0); // gira 10 rad para frente
  wb_motor_set_position(right_motor, 10.0); // idem no motor direito
  while (wb_robot_step(TIME_STEP) != -1);  // loop principal da simulacao
  wb_robot_cleanup(); // libera recursos
  return 0;
}
```

# Compilando um controlador

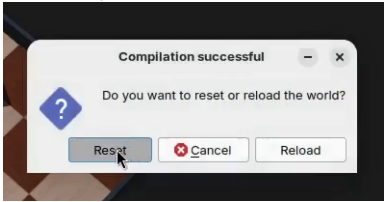
1. Salve o código novo;




1. Clique em Build;

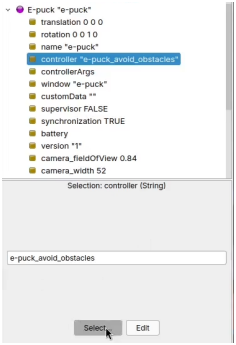



1. Clique em Reset.

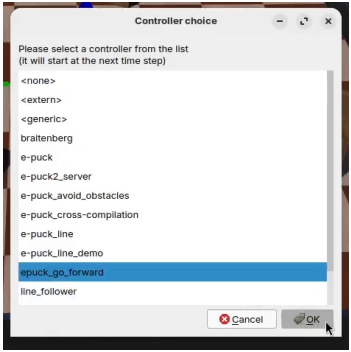


# Executando o controlador recém criado

1. Clique no robô e selecione “controller “e-puck_avoid_obstacles””;



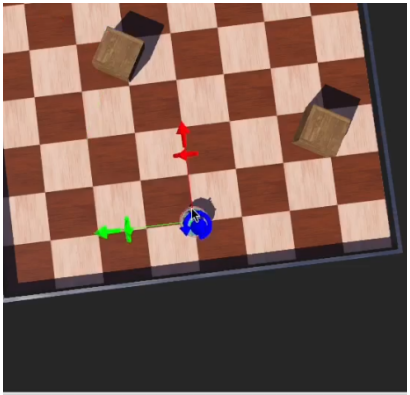

2. Selecione o controlador que criamos epuck_go_forward.

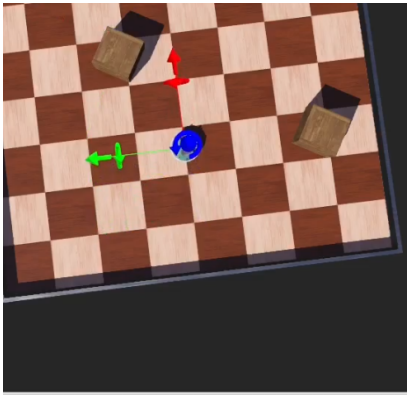



Execute a simulação e você verá o robô funcionando. É esperado que o robô percorra uma pequena distância e pare em seguida.

# Executando o controlador recém criado

Execute a simulação e você verá o robô funcionando. É esperado que o robô percorra uma pequena distância e pare em seguida.





Agora vamos colocar uma distância indefinida para que o nosso robô ande sem parar e também colocaremos um limitador de velocidade.

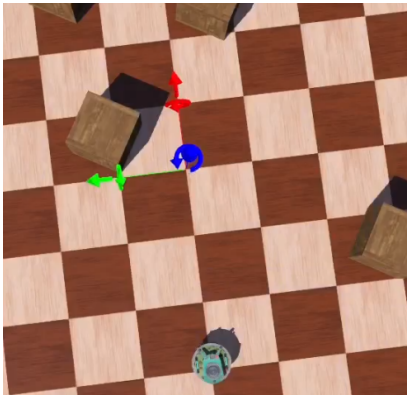
# Controle por Velocidade

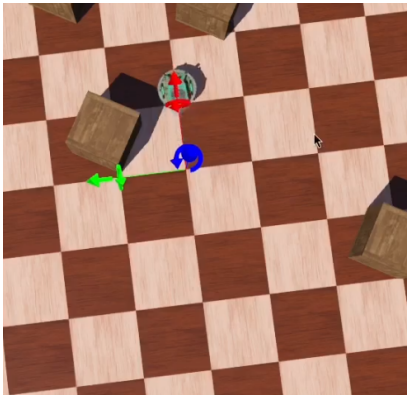
Substitua o código anterior pelo código abaixo, neste código, é possível definir a velocidade em que cada motor vai girar. É necessário clicar em Build novamente.

```c
#include <webots/robot.h> // biblioteca principal do Webots
#include <webots/motor.h> // funcoes de controle dos motores
#define TIME_STEP 64      // intervalo de tempo de cada passo (ms)
#define MAX_SPEED 6.28    // variavel global de velocide maxima
int main(int argc, char **argv) {
  wb_robot_init(); // inicializa o robo
  WbDeviceTag left_motor  = wb_robot_get_device("left wheel motor");
  WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
  // Define o quanto o motor deve girar como infinito
  wb_motor_set_position(left_motor,  INFINITY);
  wb_motor_set_position(right_motor, INFINITY);
  // Dez por cento da velocidade maxima definida
  wb_motor_set_velocity(left_motor,  0.1 * MAX_SPEED);
  wb_motor_set_velocity(right_motor, 0.1 * MAX_SPEED);
  while (wb_robot_step(TIME_STEP) != -1);  // loop principal da simulacao
  wb_robot_cleanup(); // libera recursos
  return 0;
}
```

# Controle por Velocidade

Clique em build novamente e execute a simulação. O robô andará em linha reta indefinidamente até bater em uma caixa ou na parede da arena. Além de estar a uma velocidade de 10% da velocidade máxima.





# Agradecimentos

Este trabalho foi realizado com o apoio da Pró-Reitoria de Graduação (PROGRAD), do Laboratório Machine Learning and Robotics (MLBots) e do Departamento de Computação (DECOM) da Universidade Federal de Ouro Preto (UFOP).

Leonardo de Souza Gomes, leonardo.sg@aluno.ufop.br

Prof. Dr. André Luiz C. Ottoni, andre.ottoni@ufop.edu.br

Edital PROGRAD nº 27, de 26 de maio de 2026

# Referências


[1] Cyberbotics, *Tutorial 1: Your First Simulation in Webots*, https://cyberbotics.com/doc/guide/tutorial-1-your-first-simulation-in-webots?tab-language=c, Acesso em: 3 ago. 2026, 2026. (acesso em 03/08/2026).

[2] Cyberbotics, *The User Interfaces*, https://cyberbotics.com/doc/guide/the-user-interface, Acesso em: 12 ago. 2026, 2026. (acesso em 12/08/2026).